RAG Learning

# 4.4 Theory of Computation

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.4 is the most abstract — and most examined — part of the AQA specification. It asks *what can a computer, in principle, compute?* and *how efficiently?* The tools are formal: abstract machines (finite-state machines and Turing machines), formal languages (regular expressions and Backus–Naur Form), and complexity theory (Big-O, P vs NP, and the halting problem). Examiners reward precise definitions, accurate notation, and the ability to *design* small machines or grammars to solve a stated problem. This note works each spec section 4.4.1–4.4.17 in order with worked traces you can reproduce in an exam.

## 4.4.1 Abstraction and Automation

Computation is the application of **abstraction** followed by **automation**. Abstraction strips away irrelevant detail to produce a solvable model; automation executes the abstracted solution on a computer.

> **Abstraction** is the process of removing unnecessary detail from a problem so that the underlying structure can be reasoned about. The result of abstraction is a *model* — a simplified representation of the real-world situation.

> **Automation** is the process of creating a model, devising an algorithm that manipulates that model, and then running the algorithm on a computer. Automation is the *execution* half of computation; abstraction is the *modelling* half.

Two flavours of abstraction underpin everything in 4.4:

- **Representational abstraction** — deciding what to *keep* about the real world. A London Underground map keeps station names and lines but discards geographic accuracy because the

traveller does not need it.

- **Procedural abstraction** — hiding the *how* of a process behind a named interface. A call to `sort(list)` abstracts away whether the implementation is bubble sort or Timsort.

## 4.4.2 Types of Abstraction

The AQA spec enumerates six distinct kinds of abstraction. Each strips a different dimension of detail.

| Abstraction type | What it hides | Example |
|---|---|---|
| Generalisation / categorisation | Specific differences between individuals; groups them by shared features. | All *Spaniels*, *Poodles*, *Labradors* abstracted as `Dog`; all `Dog`, `Cat`, `Horse` abstracted as `Mammal`. |
| Representational | Physical reality; keeps only the features relevant to the task. | Tube map; circuit diagram; architectural floor plan. |
| Procedural | The step-by-step implementation of a process. | Calling `print(x)` without knowing how the OS drives the graphics card. |
| Functional | State, side-effects, and how a mapping is computed — only the input→output relation matters. | A pure mathematical function `f(x)` = $x^2$. No memory of previous calls. |
| Data | The physical storage layout of a value. | A `Stack` ADT exposes push/pop but hides whether the stack is an array or a linked list. |
| Problem | Aspects of a real-world problem irrelevant to solving it; sometimes *reduces* it to a known one. | Google Maps abstracts road traffic into a weighted graph so Dijkstra can be applied. |

**Exam Tip:** AQA routinely asks "name two types of abstraction and give an example of each" for 4 marks. Always pair the *named type* with a *concrete example* — just listing types without

examples typically earns only half marks.

## 4.4.3 Information Hiding

**Information hiding** is the principle that a module or component exposes only the details a client needs, concealing the rest. It is the mechanism by which abstractions are enforced: clients cannot (and must not) depend on hidden internals, so those internals can change freely.

Benefits of information hiding:

- **Decoupling:** internal changes do not break clients, because clients depend only on the public interface.
- **Easier reasoning:** a programmer can understand one module at a time without holding the rest of the program in their head.
- **Security:** sensitive state cannot be tampered with directly — it can only be mutated via sanctioned operations.
- **Substitutability:** any implementation conforming to the interface is interchangeable (e.g. swap `ArrayList` for `LinkedList`).

Concrete examples: `private` fields in Java/C#; public API versus implementation files in C (`.h` vs `.c`); the OS kernel exposing system calls while hiding page tables; the database server exposing SQL while hiding B-trees on disk.

## 4.4.4 Procedural Abstraction

Procedural abstraction packages a sequence of steps behind a named *procedure* or *subroutine*. The caller supplies arguments and cares only about the result — not how it is produced.

```
FUNCTION averageTemperature(readings) RETURNS Real
    total ← 0
    FOR EACH r IN readings
        total ← total + r
    ENDFOR
```

```
    RETURN total / LENGTH(readings)
ENDFUNCTION
```

A weather-station programmer can now invoke `averageTemperature(todaysReadings)` without re-deriving the mean. The *procedure* has abstracted the summation loop. Procedural abstraction is what makes programs larger than a few hundred lines manageable.

## 4.4.5 Functional Abstraction

A **functional abstraction** goes further than procedural: the procedure is treated as a *mathematical function*, returning the same output for the same input with *no side effects*. Internal state, the order of calls, and hidden I/O are all concealed.

```
FUNCTION square(x) RETURNS Integer
    RETURN x * x
ENDFUNCTION
```

Contrast with a procedure that *prints* a square: printing is a side-effect, so it is not a functional abstraction. Functional abstractions compose cleanly: `square(add(3, 4))` evaluates purely from inputs to outputs. This purity is what lets compilers memoise, parallelise, and reorder calls safely.

## 4.4.6 Data Abstraction

> A **data abstraction** (also called an *abstract data type*, ADT) defines a set of values and the operations on them *without committing to how they are stored*. Users of the ADT interact through its operations only; the implementation is hidden.

Examples drawn from 4.2: a `Stack` exposes `push`, `pop`, `peek`, `isEmpty`; a `Queue` exposes `enQueue`, `deQueue`, `isEmpty`; a `Dictionary` exposes `get(key)`, `put(key, value)`, `remove(key)`. In each case the backing storage (array, linked list, hash table, tree) is an implementation detail that can be swapped without clients noticing.

## 4.4.7 Problem Abstraction / Reduction

**Problem abstraction** removes real-world clutter until only the computational essence remains. **Problem reduction** goes further: it transforms the new problem into an *already-solved* one. Reduction is a core technique in complexity theory (§4.4.16).

- Routing a delivery van → shortest path in a weighted graph → Dijkstra.
- Allocating exam rooms to candidates → graph colouring.
- Scheduling university lectures with non-clashing prerequisites → topological sort on a DAG.
- Finding a password of length 8 → searching a space of $62^8$ strings → brute force / dictionary attack.

## 4.4.8 Decomposition

> **Decomposition** is breaking a large problem into smaller, more manageable sub-problems that can be solved independently. Each sub-solution is then combined to solve the original problem. Decomposition is the companion of abstraction and the backbone of top-down design.

Example — designing a school management system. Top-level decomposition:

- Student records (admissions, attendance, reports).
- Timetabling (teacher allocation, room allocation, class scheduling).
- Finance (fees, payroll, procurement).
- Communication (email to parents, SMS, newsletter).

Each component decomposes further — "attendance" into registration, absence tracking, reporting — until each leaf is a single cohesive module small enough for a single programmer or subroutine.

## 4.4.9 Composition

**Composition** is the inverse of decomposition: combining small, well-understood components to build larger ones. Modern software engineering relies on composition — libraries, frameworks, and microservices are composed rather than rewritten.

```
FUNCTION studentAverage(name)
    readings ← loadGradesFromDB(name)    // procedural abstraction
    RETURN mean(readings)                   // functional abstraction
ENDFUNCTION
```

Here `studentAverage` is composed from two independent abstractions. Composition works only because each component hides its internals (information hiding) and declares a stable interface (data/procedural abstraction).

## 4.4.10 Automation

Once abstraction has produced a model and decomposition has produced sub-models, **automation** executes them on a computer. The canonical pipeline:

1. **Model** the problem (abstraction + decomposition).
2. **Devise an algorithm** that manipulates the model (pseudocode).
3. **Implement** the algorithm in a programming language (translation).
4. **Execute** the implementation on a computer (automation proper).
5. **Interpret** the results back in terms of the real world (removing the abstraction).

Automation turns repeated manual tasks into algorithmic ones — payroll, weather forecasting, chess play, self-driving — all begin with an abstraction and end with an algorithm run on silicon.

## 4.4.11 Finite State Machines (FSMs)

> A **finite state machine** (FSM) is an abstract machine consisting of a finite set of *states*, one designated *start state*, a set of *accepting* (final) states, an *input alphabet*, and a *transition function* that, given a current state and input symbol, determines the next state.

Formally a **deterministic finite automaton** (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

- $Q$ — finite set of states;
- $\Sigma$ — input alphabet (finite set of symbols);
- $\delta : Q \times \Sigma \to Q$ — the transition function (exactly one next state per input);
- $q_0 \in Q$ — the start state;
- $F \subseteq Q$ — the set of accepting states.

> A **non-deterministic finite automaton** (NFA) relaxes the transition function to $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \wp(Q)$ — for the same input a state may transition to *zero, one, or many* next states, and ε-moves (empty-input transitions) are permitted. Every NFA can be converted to an equivalent DFA (potentially with exponentially more states) — they accept exactly the same class of languages.

## FSMs with Output — Mealy and Moore

The FSMs above are *acceptors*: they only answer yes/no. An FSM with output is a *transducer*:

> A **Mealy machine** produces output on each *transition* — the output depends on the current state *and* the current input.

> A **Moore machine** produces output from each *state* — the output depends on the current state only, not the input.

| Aspect | Mealy | Moore |
|---|---|---|
| Output attached to | transitions (edges) | states (nodes) |
| Reacts to input | immediately in the same step | on the *next* step |
| Typically needs | fewer states | more states (one per distinct output) |
| Used in | fast synchronous circuits | situations where output stability matters |

## Worked Example 1 — Vending Machine (Mealy)

A drink costs 30p. The machine accepts 10p and 20p coins and dispenses when the running total reaches 30p.

```
States: {S0, S10, S20}        (running total inserted)
Alphabet: {10, 20}
Start state: S0
Accepting state: S0 (after a successful dispense we are back to empty)

Transitions (input / output):
    S0  --10/nothing--> S10
    S0  --20/nothing--> S20
    S10 --10/nothing--> S20
    S10 --20/dispense--> S0
    S20 --10/dispense--> S0
    S20 --20/dispense+10p change--> S0
```

## Worked Example 2 — DFA for Binary Strings Divisible by 3

A classic AQA-style question: design an FSM that accepts exactly the binary strings whose value is divisible by 3. The key insight is that the *state* can represent the current *remainder mod 3*.

If the current number has remainder `r` and we append bit `b`, the new number is `2r + b`, so the new remainder is `(2r + b) mod 3`.

```
States: {q0, q1, q2}  representing remainder 0, 1, 2
Alphabet: {0, 1}
Start state: q0  (empty string ↔ value 0)
Accepting state: {q0}

Transition function δ:
    δ(q0, 0) = q0     (0 · 2 + 0 = 0 mod 3 = 0)
    δ(q0, 1) = q1     (0 · 2 + 1 = 1 mod 3 = 1)
    δ(q1, 0) = q2     (1 · 2 + 0 = 2 mod 3 = 2)
    δ(q1, 1) = q0     (1 · 2 + 1 = 3 mod 3 = 0)
    δ(q2, 0) = q1     (2 · 2 + 0 = 4 mod 3 = 1)
    δ(q2, 1) = q2     (2 · 2 + 1 = 5 mod 3 = 2)
```

© RAG Learning · Not for redistribution

## State-Transition Table

| State | On input 0 | On input 1 | Accepting? |
|---|---|---|---|
| → q0 | q0 | q1 | Yes |
| q1 | q2 | q0 | No |
| q2 | q1 | q2 | No |

## State-Transition Diagram (textual)

```
      0
     ↻
--> ( q0 ) <======= accepting
     | \
     | 1
     v
    ( q1 )  <--- 1
   /  ^    \
  0   |     \  1 --> back to q0
  v   |      \
 ( q2 ) ↻ 1
     ^
     |  0
     |
    (q1)
```

A cleaner textual description: from `q0`, a 0 is a self-loop; a 1 goes to `q1`. From `q1`, a 0 goes to `q2`; a 1 goes back to `q0`. From `q2`, a 0 goes to `q1`; a 1 is a self-loop.

## Step-by-Step Trace on Input $1101_2$ (= 13)

We process the bits left-to-right. 13 mod 3 = 1, so we expect the FSM to *reject* (end in q1, not q0).

| Step | Current state | Read bit | Value so far | Next state |
|---|---|---|---|---|
| 0 | q0 | — | (empty, 0) | q0 |

| | | | | |
|---|---|---|---|---|
| 1 | q0 | 1 | 1 | q1 |
| 2 | q1 | 1 | 3 | q0 |
| 3 | q0 | 0 | 6 | q0 |
| 4 | q0 | 1 | 13 | q1 |

Final state = **q1**, which is *not* accepting — so the FSM correctly rejects $1101_2$ = 13. Trace $110_2$ = 6: q0 → q1 → q0 → q0. Final state q0 is accepting — correct (6 mod 3 = 0).

> **Exam Tip:** For FSM exam questions, always produce *both* the transition diagram and the transition table. Label the start state with an arrow and accepting states with double circles (or clearly annotate "accepting"). Missing annotations lose easy marks.

## 4.4.12 Turing Machines

> A **Turing machine** (TM) is an abstract model of computation with: (i) an *infinite tape* divided into cells, each holding one symbol from a tape alphabet; (ii) a *read/write head* positioned over one cell at a time; (iii) a finite set of states with a designated start state and accepting state(s); and (iv) a *transition function* that, given (current state, symbol under head), specifies (symbol to write, direction to move: L or R, next state).

Formally a TM is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$:

- $Q$ — finite set of states;
- $\Sigma$ — input alphabet (does *not* contain the blank symbol ⊔);
- $\Gamma$ — tape alphabet, with $\Sigma \subseteq \Gamma$ and $\sqcup \in \Gamma$;
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ — the transition function;
- $q_0$ — start state;
- $q_{accept}$ — accepting halt state;

- `q_reject` — rejecting halt state (distinct from $q_{accept}$).

Unlike an FSM, a Turing machine has **unbounded memory** (the infinite tape) and can revisit cells. This makes it strictly more powerful than any FSM: a TM can recognise non-regular languages such as $\{0^n1^n : n \geq 0\}$ or balanced-bracket languages, which no FSM can.

## Universal Turing Machine

> A **Universal Turing Machine** (UTM) is a single Turing machine that, given on its tape an encoding of *any* Turing machine M and an input w, simulates M running on w. The UTM is the mathematical ancestor of the stored-program computer: one machine, capable of computing anything computable, parameterised by a program on the tape.

## The Church–Turing Thesis

> The **Church–Turing thesis** asserts that any function that is *intuitively* computable by a mechanical procedure can be computed by a Turing machine. Equivalently: Turing machines, λ-calculus, partial-recursive functions, and all realistic models of computation are *equivalent* in computing power. The thesis is not a mathematical theorem but a foundational working assumption of computer science.

A language or device is **Turing-complete** if it can simulate a Universal Turing Machine. Python, C, Java, JavaScript, the λ-calculus, Conway's Game of Life, even Magic: The Gathering's combat rules are all known to be Turing-complete. Any two Turing-complete systems compute exactly the same set of functions (though they may take vastly different amounts of time).

## Worked Example — TM that Adds 1 to a Binary Number

Goal: given a binary number written MSB-first on the tape (followed by blanks), write back the number + 1. The trick is to scan to the right end, then *propagate a carry* leftwards: turn trailing 1s into 0s, and the first 0 encountered into a 1. If the whole number is 1s, extend the tape.

```
States: {q0, q1, q2, q_accept}
Tape alphabet: {0, 1, ⊔}
Start state: q0   (scan right to the blank)
```

```
Accepting state: q_accept

Transition function δ:
    δ(q0, 0) = (q0, 0, R)        // stay in q0, move right
    δ(q0, 1) = (q0, 1, R)
    δ(q0, ⊔) = (q1, ⊔, L)     // hit end, switch to carry mode, move left

    δ(q1, 0) = (q2, 1, L)        // first 0: flip to 1, done adding
    δ(q1, 1) = (q1, 0, L)        // trailing 1: flip to 0, keep carrying
    δ(q1, ⊔) = (q2, 1, L)      // overflow: write 1 into new leading position

    δ(q2, 0) = (q2, 0, L)        // scan left to start
    δ(q2, 1) = (q2, 1, L)
    δ(q2, ⊔) = (q_accept, ⊔, R)  // at left end, halt and accept
```

## Step-by-Step Trace on Input 11

Tape layout notation: the head position is shown by square brackets around a cell. Blanks are shown as `_`.

| Step | State | Tape | Read | Action |
|---|---|---|---|---|
| 0 | q0 | [1] 1 _ _ | 1 | write 1, move R, stay q0 |
| 1 | q0 | 1 [1] _ _ | 1 | write 1, move R, stay q0 |
| 2 | q0 | 1 1 [_] _ | _ | write _, move L, go q1 |
| 3 | q1 | 1 [1] _ _ | 1 | write 0, move L, stay q1 |
| 4 | q1 | [1] 0 _ _ | 1 | write 0, move L, stay q1 |
| 5 | q1 | [_] 0 0 _ | _ | write 1, move L, go q2 |
| 6 | q2 | [_] 1 0 0 _ | _ | halt in q_accept |

Final tape: `1 0 0` = 4 in decimal. Correct: $11_2$ = 3, and 3 + 1 = 4. The tape *grew* leftwards to accommodate the overflow, which a finite-state machine could never do.

### FSM vs Turing Machine

| Property | Finite State Machine | Turing Machine |
|---|---|---|
| Memory | None beyond the current state | Infinite tape — unbounded read/write memory |
| Input access | One pass, left-to-right | Head can re-read and overwrite any cell |
| Recognises | Regular languages | Recursively enumerable languages |
| Example language | Strings ending in 01 | $\{0^n1^n : n \geq 0\}$; balanced brackets; primes |
| Halting | Always halts after reading input | May loop forever (cf. halting problem) |
| Equivalent to | Regular expressions | Any general-purpose programming language |

**Exam Tip:** When an exam asks "why is a Turing machine more powerful than an FSM?", give a two-part answer: (1) the tape provides *unbounded memory*, and (2) the head can *revisit* and *overwrite* cells. Supplement with a concrete example — $\{0^n1^n\}$ cannot be recognised by any FSM because the FSM would need to count, which requires unbounded memory.

## 4.4.13 Regular Expressions

A **regular expression** (regex) is a compact notation for describing a *regular language* — exactly the class of languages recognised by an FSM (DFA or NFA). Every regex corresponds to an FSM and vice-versa.

### Regex Metacharacter Cheat Sheet

| Metacharacter | Meaning | Example | Matches |
|---|---|---|---|
| `.` | any single character (except newline) | `c.t` | `cat`, `cot`, `c5t` |
| `*` | zero or more of preceding | `ab*` | `a`, `ab`, `abbbb` |
| `+` | one or more of preceding | `ab+` | `ab`, `abbb` (but not `a`) |
| `?` | zero or one of preceding (optional) | `colou?r` | `color`, `colour` |
| `[…]` | character class — any one listed | `[aeiou]` | any single vowel |
| `[^…]` | negated character class | `[^0-9]` | any single non-digit |
| `(…)` | grouping (and capture) | `(ab)+` | `ab`, `abab`, `ababab` |
| `\|` | alternation (either/or) | `cat\|dog` | `cat` or `dog` |
| `^` | start of string anchor | `^Hello` | string starting `Hello` |
| `$` | end of string anchor | `end$` | string ending `end` |
| `\d` | digit shortcut; same as `[0-9]` | `\d\d` | two digits |

## Worked Regex Examples

```
Pattern: ^-?\d+$                  (integer with optional sign)
Matches:    "42", "-7", "0", "12345"
Rejects:    "", "3.14", "+5" (no + allowed), "abc", "12a"

Pattern: ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$    (email-like)
Matches:    "alice@example.com", "bob.smith@uni.ac.uk"
Rejects:    "no-at-sign.com", "@nouser.com", "user@.com"

Pattern: ^\d{2}/\d{2}/\d{4}$       (DD/MM/YYYY date shape)
Matches:    "21/04/2026", "01/01/2000"
Rejects:    "2026-04-21", "1/4/2026", "21/4/26"

Pattern: (ab)+                     (one or more 'ab')
```

```
Matches:    "ab", "abab", "ababab"
Rejects:    "", "a", "b", "aabb", "aba" (trailing a is unmatched)
```

### Regex ≡ Regular Languages ≡ DFA / NFA

Kleene's theorem establishes a three-way equivalence. For any:

- Regex, there is an equivalent NFA (Thompson's construction).
- NFA, there is an equivalent DFA (the subset construction).
- DFA, there is an equivalent regex (state elimination).

So "regular language", "language accepted by some DFA", and "language described by some regex" denote exactly the same set of languages. This is why a regex *cannot* describe balanced parentheses, palindromes, or $\{0^n1^n\}$ — none of these is regular.

> **Exam Tip:** AQA often asks you to write a regex *and* construct the equivalent FSM. Practise going both directions: regex → NFA by Thompson, and small DFA → regex by inspection. Remember regex cannot count unboundedly, so balanced brackets require a BNF grammar (next section) — not a regex.

## 4.4.14 Backus–Naur Form (BNF)

> **Backus–Naur Form** (BNF) is a notation for formally specifying a *context-free grammar*. A BNF grammar consists of *non-terminals* (surrounded by angle brackets), *terminals* (literal symbols), and *production rules* of the form `<LHS> ::= RHS₁ | RHS₂ | …`. BNF is strictly more expressive than regex — it can describe balanced parentheses and other recursive structures.

### Components

- **Non-terminal**: a named variable in the grammar, enclosed in `<>`. Non-terminals are *expanded* using productions.
- **Terminal**: a literal symbol that appears in the final string — e.g. `0`, `+`, `if`.

- **Production rule**: `<non-terminal> ::=` ... defines how the LHS can be rewritten.
- `|` : alternation — "or".
- **Recursion**: a rule that refers to itself (directly or indirectly). Recursion is what gives BNF its extra power over regex.

## Worked Example 1 — BNF for Simple Arithmetic Expressions

A grammar for integer arithmetic expressions with +, −, *, / and parentheses, honouring precedence (* and / bind tighter than + and −):

```
<expr>    ::= <term> | <expr> "+" <term> | <expr> "-" <term>
<term>    ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor>  ::= <number> | "(" <expr> ")"
<number>  ::= <digit> | <digit> <number>
<digit>   ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

A derivation of `3 + 4 * 2`:

```
<expr>
  ⇒ <expr> "+" <term>
  ⇒ <term> "+" <term>
  ⇒ <factor> "+" <term>
  ⇒ <number> "+" <term>
  ⇒ <digit> "+" <term>
  ⇒ "3" "+" <term>
  ⇒ "3" "+" <term> "*" <factor>
  ⇒ "3" "+" <factor> "*" <factor>
  ⇒ "3" "+" "4" "*" "2"
```

Precedence falls out of the grammar structure: `<term>` sits *inside* `<expr>`, so multiplication is always grouped before addition.

## Worked Example 2 — BNF for a UK Postcode

UK postcodes have the form `AREA DISTRICT SPACE SECTOR UNIT`, e.g. `SW1A 1AA` or `M1 1AE`.

```
<postcode>    ::= <outward> " " <inward>
<outward>     ::= <area> <district>
```

```
<area>        ::= <letter> | <letter> <letter>
<district>    ::= <digit> | <digit> <digit> | <digit> <letter>
<inward>      ::= <digit> <letter> <letter>
<letter>      ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J"
                | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T"
                | "U" | "V" | "W" | "X" | "Y" | "Z"
<digit>       ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Derivation of `M1 1AE`: `<postcode> ⇒ <outward> " " <inward> ⇒ <area><district> " " <inward> ⇒ "M" "1" " " "1" "A" "E"`.

### Why BNF Beats Regex

Consider the language of balanced parentheses `{ "", "()", "(())", "()()", "((()))", … }`. The BNF grammar is a one-liner:

```
<brackets> ::= "" | "(" <brackets> ")" <brackets>
```

No regular expression can describe this language, because recognising balanced brackets requires *counting arbitrarily deep nesting*, which no DFA (and therefore no regex) can do. Recursion in BNF provides the extra expressive power.

> **Exam Tip:** BNF exam questions usually ask "derive this string from the grammar" or "write a BNF rule for…". Always present derivations as a single chain with `⇒` between steps, underline or bold the non-terminal being rewritten, and stop only when the string consists entirely of terminals.

## 4.4.15 Big-O Notation

> **Big-O notation** describes an *asymptotic upper bound* on a function's growth. $f(n) = O(g(n))$ means there exist constants $c > 0$ and $n_0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$. We use Big-O to talk about the worst-case resources (usually time or space) an algorithm needs as input size grows.

### Why Constants are Dropped

Big-O hides both constant factors and lower-order terms because, as `n → ∞`, they become irrelevant. For example `3n² + 5n + 10 = O(n²)` — the `5n + 10` is dominated by `n²` for large n, and the constant 3 disappears into the free constant c of the definition. Dropping constants lets us compare *asymptotic shapes* of growth without being distracted by machine-dependent multipliers.

### Growth-Rate Comparison

From best to worst:

```
O(1)  <  O(log n)  <  O(n)  <  O(n log n)  <  O(n²)  <  O(n³)  <  O(2ⁿ)  <  O(n!)
```

| Class | n = 10 | n = 100 | n = 1000 | Example |
|---|---|---|---|---|
| O(1) | 1 | 1 | 1 | Array indexing |
| O(log n) | 3 | 7 | 10 | Binary search |
| O(n) | 10 | 100 | 1 000 | Linear search |
| O(n log n) | 33 | 664 | 9 966 | Merge sort |
| $O(n^2)$ | 100 | 10 000 | $10^6$ | Bubble sort |
| $O(2^n)$ | 1 024 | $\approx 10^{30}$ | $\approx 10^{301}$ | Naive subset enumeration |
| O(n!) | 3.6 M | $\approx 10^{158}$ | intractable | Brute-force TSP |

Note how the last two lines pass astronomical numbers before `n` reaches 100 — this is the *exponential wall* that separates tractable from intractable.

## 4.4.16 Classification of Algorithms

### Tractable vs Intractable

> A problem is **tractable** if it has an algorithm whose worst-case running time is bounded by a *polynomial* in the input size — $O(n^k)$ for some constant k. A problem is **intractable** if no such polynomial-time algorithm is known; typically the best known algorithm takes exponential or worse time.

The distinction is practical rather than absolute — an $O(n^{100})$ algorithm is "tractable" in theory but useless in practice. In exam terms, **polynomial = tractable, exponential = intractable**.

## Decision Problems and Complexity Classes

A **decision problem** is a problem whose answer is yes or no. Complexity classes are defined in terms of decision problems.

> **P** is the class of decision problems *solvable* by a deterministic Turing machine in polynomial time. Intuitively: problems a computer can solve in a reasonable amount of time.

> **NP** is the class of decision problems whose *solutions can be verified* by a deterministic Turing machine in polynomial time, given a certificate. Equivalently: problems solvable by a non-deterministic Turing machine in polynomial time.

Clearly $P \subseteq NP$ (if you can solve in poly time, you can verify in poly time — just solve it again and compare). The **P vs NP question** asks whether the inclusion is strict: is there a problem in NP that is not in P? The consensus is P ≠ NP, but no one has proved it — a US$1 million Clay Millennium Prize awaits the first proof.

## NP-Hard and NP-Complete

- A problem H is **NP-hard** if every problem in NP can be reduced to H in polynomial time. NP-hard problems are "at least as hard as anything in NP".
- A problem C is **NP-complete** if C is in NP *and* C is NP-hard. NP-complete problems are the hardest problems in NP: if any one of them had a polynomial-time algorithm, then $P = NP$.

## Complexity Class Hierarchy

| Class | Definition | Example problem |
|---|---|---|
| P | Solvable in polynomial time | Sorting, shortest path (Dijkstra), matrix multiply, primality testing (since 2002) |
| NP | Verifiable in polynomial time | SAT, TSP decision, Hamiltonian path, graph 3-colouring, Sudoku |
| NP-complete | Hardest problems in NP | SAT (Cook–Levin theorem), 3-SAT, subset sum, TSP decision |
| PSPACE | Solvable with polynomial space | Generalised chess/Go; quantified boolean formulae |
| EXPTIME | Solvable in exponential time | Generalised chess (with no repetition rule) |
| Undecidable | No algorithm exists at all | Halting problem; Entscheidungsproblem |

Containment: `P ⊆ NP ⊆ PSPACE ⊆ EXPTIME`. It is known that `P ⊂ EXPTIME` (strict), so at least one of the intermediate inclusions must be strict — but which one (P≠NP? NP≠PSPACE? PSPACE≠EXPTIME?) is open.

## Canonical Examples

- **SAT (Boolean satisfiability):** given a Boolean formula, is there an assignment of variables making it true? NP-complete (Cook–Levin, 1971) — the first problem proved NP-complete, and the "mother" of all NP-hardness reductions.
- **Travelling Salesman Problem (TSP):** given a weighted graph and a budget k, is there a Hamiltonian tour of total weight ≤ k? NP-complete. The optimisation version (shortest tour) is NP-hard. Brute force is O(n!); dynamic programming (Held–Karp) is $O(n^2 2^n)$.
- **Hamiltonian path:** given a graph, does a path exist that visits every vertex exactly once? NP-complete. Contrast with Eulerian path (every *edge* exactly once), which is solvable in O(V + E).

**Exam Tip:** For P vs NP questions AQA wants three precise statements: (1) P = decidable in polynomial time; (2) NP = *verifiable* in polynomial time; (3) whether P = NP is the unresolved question. Do not confuse NP with "non-polynomial" — NP stands for *non-deterministic polynomial*.

## 4.4.17 The Halting Problem

The **Halting Problem** asks: given any program P and input w, does P halt (finish) when run on w? Turing proved in 1936 that no algorithm can solve this for all (P, w) — it is *undecidable*.

A decision problem is **decidable** if there exists a Turing machine that, on every input, halts and correctly outputs yes or no. It is **undecidable** otherwise. Undecidability is a *mathematical impossibility*, not merely a statement about current technology or cleverness.

### Turing's Proof by Contradiction

The proof is a classic diagonal argument. Suppose, for contradiction, that a halting-decider subroutine H exists:

```
FUNCTION H(program, input) RETURNS Boolean
    // Returns True  if 'program' halts on 'input'
    // Returns False if 'program' loops forever on 'input'
ENDFUNCTION
```

Now construct a new program D ("diagonaliser") that uses H to do the *opposite* of what H predicts when D is fed its own source code:

```
FUNCTION D(program)
    IF H(program, program) = True THEN
        LOOP FOREVER         // H said "halts" → D loops
    ELSE
        HALT                 // H said "loops" → D halts
```

```
    ENDIF
ENDFUNCTION
```

Now run `D(D)` — feed D its own source code as input. Two cases:

- **Case 1:** `H(D, D) = True`, meaning "D halts on D". But by D's definition, D then enters the infinite loop — so D does *not* halt on D. **Contradiction.**
- **Case 2:** `H(D, D) = False`, meaning "D does not halt on D". But by D's definition, D then hits the `HALT` — so D *does* halt on D. **Contradiction.**

Both cases are contradictions, so our assumption must be wrong. No such H can exist — the halting problem is undecidable.

### Implications

- No general *program verifier* can decide for every program whether it halts, terminates cleanly, or has any non-trivial semantic property — **Rice's theorem** generalises this to all non-trivial semantic properties of programs.
- Production static analysers (linters, type checkers, bounded model checkers) get around undecidability by being *incomplete*: they may answer "yes", "no", or "don't know", and they give up on edge cases.
- Compilers cannot always decide whether a branch is reachable; operating systems cannot always detect a user process stuck in an infinite loop versus one merely taking a long time.
- Conversely, this places a *fundamental limit* on what software engineering can automate — some bugs cannot be caught in general by any automated tool.

> **Exam Tip:** The halting-problem proof is a favourite AQA 6-8 mark extended response. Structure your answer as: (1) state the problem; (2) *assume* halting-decider H exists; (3) construct D that does the opposite of H's prediction on its own input; (4) derive contradiction in both cases of H(D, D); (5) conclude H cannot exist. Make the contradiction explicit in both branches — examiners mark the logical structure, not the prose.

## Summary of Key Definitions

| Term | Definition (one line) |
|---|---|
| Abstraction | Removing irrelevant detail to reveal essential structure. |
| Decomposition | Splitting a problem into smaller, independently solvable sub-problems. |
| Automation | Running an algorithmic model of a problem on a computer. |
| FSM / DFA | Abstract machine with finite states and a deterministic transition function. |
| NFA | FSM allowing multiple and ε-transitions; equivalent in power to DFA. |
| Mealy machine | FSM with output on transitions. |
| Moore machine | FSM with output on states. |
| Turing machine | Abstract machine with infinite tape, head, and transition function; 7-tuple. |
| Universal TM | A TM that simulates any other TM given its encoding on the tape. |
| Church–Turing thesis | All realistic computation models compute the same functions as a TM. |
| Turing-complete | A system that can simulate a Universal Turing Machine. |
| Regular expression | Notation for regular languages; equivalent to DFA/NFA. |
| BNF | Notation for context-free grammars; more expressive than regex. |
| Tractable / Intractable | Solvable in polynomial time / requires super-polynomial time. |
| P | Decision problems solvable in polynomial time. |
| NP | Decision problems whose solutions are verifiable in polynomial time. |
| NP-complete | Problems in NP that every NP problem reduces to. |
| Decidable | A TM exists that halts on every input and answers correctly. |

© RAG Learning · Not for redistribution

| | |
|---|---|
| Undecidable | No such TM exists (e.g. halting problem). |
| Halting problem | Decide whether an arbitrary program halts on an arbitrary input; undecidable. |

## Summary Checklist

- Define abstraction and distinguish representational, procedural, functional, data, problem, and categorisation varieties with examples.
- Explain information hiding and its benefits (decoupling, security, substitutability).
- Decompose a realistic problem top-down and describe composition as its inverse.
- Formally define a DFA as a 5-tuple and give its transition diagram and transition table.
- Distinguish Mealy and Moore FSMs and design a small machine (vending machine, traffic light).
- Design an FSM that recognises binary strings divisible by 3 and trace it on a given input.
- Define a Turing machine as a 7-tuple and explain why it is strictly more powerful than an FSM.
- Define a Universal Turing Machine and the Church–Turing thesis; give an example of a Turing-complete language.
- Trace a TM that adds 1 to a binary number on a small input.
- List the regex metacharacters and construct regexes for signed integers, dates, and email-like strings.
- Explain the equivalence of regex, regular languages, and DFA/NFA.
- Write a BNF grammar for arithmetic expressions and UK postcodes; give a derivation of a short string.
- Explain why BNF can describe balanced parentheses but regex cannot.
- State the formal definition of Big-O and justify why constants are dropped.
- List complexity classes from O(1) to O(n!) and name example algorithms for each.
- Distinguish tractable from intractable; polynomial from exponential.
- Define P and NP; state the P vs NP question; define NP-hard and NP-complete.
- Name SAT, TSP, and Hamiltonian path as canonical NP-complete problems.
- State and prove (by diagonal contradiction) that the halting problem is undecidable.

© RAG Learning · Not for redistribution

- Discuss the practical consequences of undecidability for program verification and static analysis.

© RAG Learning · Not for redistribution